Automated Species Identification of Lepidoptera: A Comparative Study of MLP, CNN, and ResNet Architectures

João Vaz[1] and João Francisco Morais[2]

[1] Department of Informatics Engineering, University of Coimbra, Coimbra, Portugal
[2] Department of Physics, University of Coimbra, Coimbra, Portugal
{uc2022231087,uc2022232807}@student.uc.pt

Abstract. The automation of biodiversity monitoring is a critical challenge in conservation biology, particularly for fine-grained identification of Lepidoptera species. This study implements a supervised image classification pipeline to identify 75 distinct butterfly species from a dataset of 5,199 macro-photographs. We evaluate the progression from classical to modern deep learning architectures by systematically comparing Multi-Layer Perceptrons (MLP), Convolutional Neural Networks (CNN), and Residual Neural Networks (ResNet), with experiments across different loss functions (Cross-Entropy, MultiMarginLoss) and optimizers (Adam, RMSprop, AdamW). Performance is assessed using appropriate metrics to account for class imbalance. The best-performing model is a fine-tuned ResNet50, with transfer learning proving decisive over training from scratch across all architectures.

Keywords: Image Classification · Deep Learning · Convolutional Neural Networks · Residual Networks · Transfer Learning · Biodiversity Monitoring

1 Introduction and Problem Description

Image classification has been fundamentally transformed by deep learning, replacing hand-crafted feature engineering with end-to-end trainable models. Species identification from photographs represents a particularly challenging instance, where inter-class differences can be subtle and intra-class variance is high due to lighting conditions, camera angles, and specimen variation.

This work addresses the identification of butterfly species as a 75-class supervised classification problem, motivated by a practical deployment scenario at the Science Museum of the University of Coimbra. The dataset consists of 5,199 RGB images at 224×224 pixels, exhibiting mild class imbalance: a minimum of 57 samples (*Wood Satyr*), a maximum of 105 (*Mourning Cloak*), a mean of 69.32 samples per class, and an imbalance ratio of 1.84. Labelled data is partitioned via stratified 80/20 split into 4,159 training and 1,040 validation samples, with a separate held-out test set used exclusively for the Kaggle competition leaderboard.

Three families of neural network architectures were compared: MLPs, CNNs, and ResNets, each evaluated across different layer configurations, loss functions, and optimisers. Details can be found in Section 3. Performance is evaluated using accuracy and macro-averaged F1-score, the latter preferred given the mild class imbalance, with validation loss monitored throughout training to guide early stopping and model selection. The full codebase, checkpoints, and submission scripts are publicly available.[3]

2 Approach

2.1 MLP

Three fully-connected architectures of increasing capacity were designed as a non-convolutional baseline. Input images are resized to 64×64 and flattened into 12,288-dimensional vectors via `nn.Flatten()`, discarding all spatial structure. All models use ReLU activations [9] with no output activation, as softmax is subsumed by the loss criterion where applicable. Architecture A comprises one hidden layer of 256 units, Architecture B two hidden layers of 512 and 256 units, and Architecture C three hidden layers of 1,024, 512, and 256 units. Architecture B was selected as a compromise between capacity and training cost; A and C bracket its depth. Training details are in Section 3.2.

2.2 CNN

The CNN experiments encompass four architectures trained from scratch and two initialised with ImageNet pretrained weights. *ModernLeNet* is a modernised variant of LeCun et al. [6], with two convolutional layers, batch normalisation [3], and max-pooling. *AlexNet* follows Krizhevsky et al. [5] with Local Response Normalisation omitted. *AlexNetOptimised* extends this baseline with batch normalisation after every convolutional layer and a reduced classifier head ($9{,}216 \rightarrow 512 \rightarrow 75$). *VGG16Optimised* follows Simonyan and Zisserman [12] with batch normalisation throughout and a reduced classifier, both to constrain overfitting on the small dataset. Transfer learning [14,10] is explored via *AlexNet-Pretrained* (`IMAGENET1K_V1` weights) and *VGG16Pretrained* (`vgg16_bn` weights), both with frozen feature extractors. Architecture details are summarised in Table 1.

Table 1. CNN architectures evaluated. Trainable params exclude frozen feature extractors for pretrained models.

Architecture	Classifier head	Conv	BN	Total params	Trainable	Max Ep.	Pretrained
ModernLeNet	$200{,}704 \rightarrow 512 \rightarrow 75$	2	Yes	102.8M	102.8M	30	No
AlexNet	$9{,}216 \rightarrow 4{,}096 \rightarrow 4{,}096 \rightarrow 75$	5	No	58.6M	58.6M	30	No
AlexNetOptimised	$9{,}216 \rightarrow 512 \rightarrow 75$	5	Yes	8.5M	8.5M	50	No
VGG16Optimised	$25{,}088 \rightarrow 512 \rightarrow 75$	13	Yes	27.6M	27.6M	30	No
AlexNetPretrained	$9{,}216 \rightarrow 512 \rightarrow 75$	5	No	8.5M	4.8M	30	Yes
VGG16Pretrained	$25{,}088 \rightarrow 512 \rightarrow 75$	13	Yes	27.6M	12.9M	30	Yes

[3] `https://github.com/vazeta/ACA_PROJ_1.git`

2.3 ResNet

Building on He et al. [1], ResNet architectures leverage shortcut connections to perform identity mappings, mitigating the degradation problem that arises when training very deep networks. Two strategies were explored: **custom models** (ResNet-18, -34, -50 from scratch) and **pretrained models** (ResNet-18 and -50 from `torchvision.models`).

Custom models use *BasicBlock* (two 3×3 convolutions) for ResNet-18/-34 and *BottleneckBlock* for ResNet-50. All share a common stem, *AdaptiveAvgPool2d*, Dropout ($p = 0.4$), and a fully connected output layer. Convolutional weights were initialised via He (Kaiming) normal initialisation [2], with BatchNorm parameters set to weight = 1, bias = 0. Pretrained models were adapted by replacing the classification head with Dropout ($p = 0.4$) followed by a Linear layer, and evaluated under both full *finetuning* and *frozen backbone* strategies.

Loss functions, optimisers, and learning rate schedule follow the same configuration described in Section 3.2. Custom ResNets use a learning rate of 1×10^{-3} (batch size 32) or 5×10^{-4} (batch size 16 for selected ResNet-50 runs); pretrained models use 1×10^{-4} or 5×10^{-5} accordingly. Optimisers include Adam (A) [4], AdamW (AW), and RMSprop (R). Training runs for a maximum of 40 epochs with early stopping (patience = 7) monitoring validation loss.

3 Experimental Setup

3.1 Data Preprocessing & Augmentation

RGB channel statistics for normalisation were computed over the full training set during EDA, yielding $\mu = [0.479, 0.465, 0.337]$ and $\sigma = [0.256, 0.246, 0.256]$. Scratch-trained architectures use these statistics; pretrained models use the ImageNet statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$) [8] to maintain consistency with the pretraining distribution. The MLP pipeline applies no augmentation: images are resized to 64×64 and converted to tensors without further transformation, as the flatten operation discards spatial structure that augmentation would otherwise preserve. The CNN and ResNet pipelines share an identical augmentation strategy applied exclusively during training: images undergo *RandomResizedCrop* to 224×224 with scale $(0.7, 1.0)$, *RandomHorizontalFlip* ($p = 0.5$), *RandomVerticalFlip* ($p = 0.1$), *ColorJitter* (brightness 0.3, contrast 0.3, saturation 0.2), and *RandomRotation* up to $20°$, improving generalisation [11]. Validation uses a deterministic pipeline: *Resize* to 256 followed by *CenterCrop* to 224×224.

3.2 Hyperparameters & Training Details

All CNN and ResNet experiments use `SEED=2026` for both the random state and the stratified split. The MLP pipeline was developed independently using `random_state=42`, producing a distinct but proportionally equivalent partition of the same dataset. The loss/optimiser combinations evaluated are: CE+A [4],

CE+R, MM+A, and MM+R across all three families, with CE+AW and MM+AW additionally explored in the ResNet family.

MLP Architecture B is trained for 20 epochs with a fixed learning rate of 1×10^{-3}, batch size 32, and no scheduler, early stopping, label smoothing, or weight decay.

CNN All architectures use batch size 32, weight decay 10^{-4}, R momentum 0.9, `CosineAnnealingLR` scheduling [7] with $\eta_{\min} = \text{lr} \times 10^{-2}$, early stopping with patience 7 (prioritising validation accuracy, then validation loss), and cross-entropy loss with label smoothing 0.1 [13]. The initial learning rate is 1×10^{-3} for ModernLeNet and 1×10^{-4} for all remaining architectures. AlexNetOptimised is trained for 50 epochs; all other CNN architectures for 30 epochs.

ResNet ResNet architectures use image resolution 224×224, `CosineAnnealingLR` scheduling, and early stopping with patience 7 monitoring validation loss, for a maximum of 40 epochs. The primary loss function is cross-entropy with label smoothing 0.1; selected configurations substitute MultiMarginLoss to assess its effect. Optimisers used are Adam, RMSprop (momentum 0.9), and AdamW, with weight decay 10^{-4} or 10^{-3}.

Custom ResNet-18 and ResNet-34 are evaluated with CE+A, CE+R, and MM+AW at learning rate 1×10^{-3}, batch size 32. Custom ResNet-50 includes a lower learning rate variant (5×10^{-4}, batch size 16, weight decay 10^{-3}) and MM+A/AW configurations. Pretrained fine-tuned models use 1×10^{-4} (batch size 32) or 5×10^{-5} (batch size 16, weight decay 10^{-3}) for ResNet-50; the frozen backbone variant uses 1×10^{-4}, batch size 32. ResNet-50 finetuned configurations cover CE+A, CE+R, and MM+A; ResNet-18 finetuned covers CE+A, CE+R, and MM+A; frozen variants are evaluated with CE+A only.

4 Analysis of the Results

4.1 MLP Results

Architecture B results are reported below. CE+A achieves the best performance (Acc = 0.348, F1 = 0.344), followed by CE+R (Acc = 0.300, F1 = 0.289), MM+A (Acc = 0.284, F1 = 0.274), and MM+R (Acc = 0.250, F1 = 0.240). All configurations exhibit poor absolute performance and overfitting from epoch 5 onwards, a direct consequence of the flatten operation discarding all spatial structure, the 64×64 input resolution, and the absence of regularisation techniques such as Dropout or BatchNorm. CE consistently outperforms MM across both optimisers, suggesting that margin-based supervision is less suited to high-cardinality classification than probabilistic cross-entropy.

4.2 CNN Results

From-Scratch Architectures Table 2 reports from-scratch results (SEED = 2026). ModernLeNet fails catastrophically: CE+A collapses at epoch 8 (F1 = 0.003) and remaining combinations reach at most F1 = 0.276, due to the oversized fully-connected layer (200,704 input units) causing immediate divergence. AlexNet recovers substantially, with a small train-val gap ($\leq$ 0.042) indicating well-calibrated capacity. VGG16Optimised exhibits negative train-val gaps (val > train) across all configurations, an artefact of augmentation presenting harder inputs at training time than the deterministic validation pipeline; notably, MM+A outperforms CE+A, the only CE<MM inversion observed. AlexNetOptimised achieves the best from-scratch result driven by BatchNorm stabilisation and the reduced classifier head, though at the cost of genuine overfitting (train-val gap 0.052-0.093) over 50 epochs despite early stopping.

Table 2. CNN from-scratch results (SEED = 2026), ranked by Val. Acc. OPT = optimised variant (BatchNorm + reduced head); CE = cross-entropy, MM = multi-margin; A = Adam, R = RMSprop.

Architecture	Loss	Opt.	Ep.	Train. Acc	Val. Loss	Val. Acc	Top-5 Acc	F1 Macro
AlexNetOpt	CE	A	50	0.9538	1.2633	0.8635	0.9663	0.8629
AlexNetOpt	CE	R	50	0.9315	1.3254	0.8385	0.9654	0.8350
AlexNetOpt	MM	A	50	0.8882	0.0235	0.8048	0.9625	0.8014
AlexNetOpt	MM	R	50	0.8495	0.0309	0.7971	0.9596	0.7942
AlexNet	CE	A	30	0.7735	1.6594	0.7317	0.9231	0.7271
AlexNet	CE	R	30	0.7475	1.6654	0.7212	0.9221	0.7160
VGG16Opt	MM	A	30	0.4855	0.0489	0.6865	0.9183	0.6770
AlexNet	MM	R	30	0.6319	0.0587	0.6529	0.9029	0.6434
AlexNet	MM	A	30	0.6562	0.0588	0.6500	0.9038	0.6412
VGG16Opt	CE	A	30	0.3359	2.2167	0.6163	0.9077	0.5777
VGG16Opt	CE	R	30	0.4304	2.2786	0.5577	0.8308	0.5383
VGG16Opt	MM	R	30	0.3686	0.1039	0.4894	0.8019	0.4738
ModernLeNet	CE	R	30	0.2186	3.1046	0.3154	0.6240	0.2763
ModernLeNet	MM	R	30	0.1488	0.3308	0.2106	0.4933	0.1936
ModernLeNet	MM	A	30	0.0481	0.6041	0.0894	0.2923	0.0529
ModernLeNet	CE	A	8	0.0137	4.3017	0.0221	0.0933	0.0033

Transfer Learning Table 3 reports pretrained results. CE configurations were evaluated across five seeds (mean $\pm$ std); MM at SEED = 2026 only. Both pretrained architectures surpass the best from-scratch result (F1 = 0.863) by a wide margin despite training only the classifier head. Train-val gaps are moderate and consistent (0.026-0.070) with no negative gaps: the frozen backbone eliminates the underfitting seen in VGG16 scratch, and the reduced trainable parameter budget prevents the memorisation of AlexNetOptimised. CE outperforms MM and A outperforms R across both architectures. AlexNetPretrained MM+A triggers early stopping at epoch 29, suggesting mild instability under margin-based supervision with frozen features. Val. Loss values for MM configurations reflect the bounded nature of MultiMarginLoss and are not directly comparable to CE values.

Table 3. CNN pretrained validation results ranked by Val. Acc. PT = pre-trained (ImageNet), FR = frozen backbone; CE = cross-entropy, MM = multi-margin; A = Adam, R = RMSprop. CE models: mean ± std over 5 seeds; MM models: single run (SEED = 2026).

Architecture	Loss	Opt.	Train. Acc	Val. Loss	Val. Acc	Top-5 Acc	F1 Macro
VGG16Pre	CE	A	0.9666 ± 0.0044	1.2110 ± 0.0153	0.9104 ± 0.0093	0.9869 ± 0.0051	0.9109 ± 0.0092
AlexNetPre	CE	A	0.9618 ± 0.0061	1.1976 ± 0.0184	0.9069 ± 0.0039	0.9850 ± 0.0036	0.9063 ± 0.0037
VGG16Pre	CE	R	0.9700 ± 0.0029	1.2240 ± 0.0111	0.8998 ± 0.0058	0.9787 ± 0.0053	0.9001 ± 0.0059
VGG16Pre	MM	R	0.9409	0.0146	0.8952	0.9837	0.8943
VGG16Pre	MM	A	0.9430	0.0159	0.8885	0.9798	0.8883
AlexNetPre	CE	R	0.9482 ± 0.0060	1.2512 ± 0.0209	0.8888 ± 0.0047	0.9787 ± 0.0041	0.8876 ± 0.0044
AlexNetPre	MM	R	0.9336	0.0192	0.8731	0.9769	0.8724
AlexNetPre	MM	A	0.8947	0.0132	0.8692	0.9808	0.8679

Seed Robustness Analysis Table 4 reports per-seed F1-macro for the four CE pretrained configurations across five seeds ($\{0, 1, 42, 1010, 2026\}$). VGG16Pretrained CE+A achieves the highest mean but also the largest variance, driven by a weaker run at seed 2026 (F1 = 0.898). AlexNetPretrained CE+A is the most stable configuration, with no seed deviating more than 0.010 from the mean, making it the preferred choice for deployment under variable initialisation conditions.

Table 4. Multi-seed F1-macro ($\times 5$ seeds: $\{0, 1, 42, 1010, 2026\}$). PT = pre-trained (ImageNet); CE = cross-entropy; A = Adam, R = RMSprop.

Model	s=0	s=1	s=42	s=1010	s=2026	Mean	Std
VGG16Pre CE+A	0.906	0.912	0.922	0.916	0.898	**0.911**	0.009
AlexNetPre CE+A	0.908	0.911	0.901	0.906	0.905	**0.906**	0.004
VGG16Pre CE+R	0.907	0.902	0.903	0.894	0.894	0.900	0.006
AlexNetPre CE+R	0.892	0.891	0.885	0.889	0.881	0.888	0.004

Generalisation to the Test Set Table 5 reports Kaggle leaderboard scores. The ranking is broadly consistent with validation, with pretrained models occupying all top positions. VGG16Pretrained CE+A achieves the best test accuracy, improving over its validation mean by +0.004 and confirming strong generalisation. Notably, AlexNetPretrained CE+A, the most stable model on validation, drops to rank 4 on test (0.898 ± 0.008), a -0.008 degradation suggesting its lower variance does not fully translate to the held-out distribution. VGG16Pre occupies the top two positions, indicating the deeper frozen backbone generalises more robustly. From-scratch architectures retain their relative ordering from validation, with AlexNetOptimised (rank 9) remaining the strongest non-pretrained model.

Table 5. CNN test accuracy (Kaggle leaderboard), ranked by test accuracy (mean ± std across seeds). PT = pre-trained (ImageNet), CU = custom (from scratch), OPT = custom-optimised; CE = cross-entropy, MM = multi-margin; A = Adam, R = RMSprop.

Rank	Architecture	Loss	Opt.	Seeds	Test Acc.
1	VGG16Pre	CE	A	5	0.9143 ± 0.0050
2	VGG16Pre	CE	R	5	0.9048 ± 0.0048
3	VGG16Pre	MM	R	1	0.8992
4	AlexNetPre	CE	A	5	0.8983 ± 0.0082
5	VGG16Pre	MM	A	1	0.8931
6	AlexNetPre	CE	R	5	0.8895 ± 0.0085
7	AlexNetPre	MM	R	1	0.8815
8	AlexNetPre	MM	A	1	0.8777
9	AlexNetOpt	CE	A	1	0.8738
10	AlexNetOpt	CE	R	1	0.8646
11	AlexNetOpt	MM	A	1	0.8254
12	AlexNetOpt	MM	R	1	0.8123
13	AlexNet	CE	A	1	0.7431
14	AlexNet	CE	R	1	0.7392
15	VGG16Opt	MM	A	1	0.6854
16	AlexNet	MM	R	1	0.6577
17	AlexNet	MM	A	1	0.6515
18	VGG16Opt	CE	A	1	0.6008
19	VGG16Opt	CE	R	1	0.5446
20	VGG16Opt	MM	R	1	0.5162
21	ModernLeNet	CE	R	1	0.3031
22	ModernLeNet	MM	R	1	0.2338
23	ModernLeNet	MM	A	1	0.0815
24	ModernLeNet	CE	A	1	0.0208

4.3 ResNet Results

Table 6 reports the full ranking sorted by mean validation accuracy.

Table 6. ResNet configuration ranking by validation accuracy (mean ± std across seeds, best epoch per seed by minimum val. loss). PT = pre-trained (ImageNet), CU = custom (from scratch), FR = frozen backbone; CE = cross-entropy, MM = multi-margin; A = Adam, AW = AdamW, R = RMSprop.

Rank	Configuration	Seeds	Train. Acc	Val. Loss	Val. Acc	Top-5 Acc	F1 Macro
1	PT-R50-CE-A (bs32, lr1e-4, wd1e-4)	5	0.9957 ± 0.0030	0.9379 ± 0.0291	0.9498 ± 0.0100	0.9892 ± 0.0047	0.9499 ± 0.0105
2	PT-R50-CE-A (bs16, lr5e-5, wd1e-3)	5	0.9911 ± 0.0054	0.9478 ± 0.0235	0.9483 ± 0.0063	0.9910 ± 0.0036	0.9470 ± 0.0071
3	PT-R18-CE-A (bs32, lr1e-4, wd1e-4)	5	0.9968 ± 0.0008	0.9830 ± 0.0204	0.9462 ± 0.0086	0.9908 ± 0.0035	0.9463 ± 0.0084
4	PT-R50-CE-R (bs32, lr1e-4, wd1e-4)	3	0.9980 ± 0.0005	0.9948 ± 0.0327	0.9369 ± 0.0170	0.9888 ± 0.0031	0.9367 ± 0.0181
5	PT-R50-MM-A (bs32, lr1e-4, wd1e-4)	3	0.9827 ± 0.0043	0.0069 ± 0.0023	0.9362 ± 0.0098	0.9926 ± 0.0044	0.9350 ± 0.0097
6	PT-R18-CE-R (bs32, lr1e-4, wd1e-4)	1	0.9937 ± 0.0000	1.0376 ± 0.0000	0.9212 ± 0.0000	0.9894 ± 0.0000	0.9206 ± 0.0000
7	PT-R18-MM-A (bs32, lr1e-4, wd1e-4)	1	0.9382 ± 0.0000	0.0091 ± 0.0000	0.9135 ± 0.0000	0.9904 ± 0.0000	0.9138 ± 0.0000
8	CU-R50-CE-A (bs32, lr1e-3, wd1e-4)	5	0.9587 ± 0.0079	1.1989 ± 0.0405	0.8925 ± 0.0141	0.9783 ± 0.0055	0.8921 ± 0.0140
9	CU-R50-CE-A (bs16, lr5e-4, wd1e-3)	5	0.9197 ± 0.0030	1.2495 ± 0.0297	0.8877 ± 0.0115	0.9775 ± 0.0071	0.8866 ± 0.0129
10	CU-R18-CE-A (bs32, lr1e-3, wd1e-4)	5	0.9531 ± 0.0030	1.2048 ± 0.0272	0.8877 ± 0.0086	0.9750 ± 0.0039	0.8867 ± 0.0087
11	CU-R34-CE-A (bs32, lr1e-3, wd1e-4)	5	0.9394 ± 0.0075	1.2006 ± 0.0324	0.8842 ± 0.0081	0.9773 ± 0.0044	0.8831 ± 0.0086
12	CU-R50-MM-A (bs32, lr1e-3, wd1e-4)	3	0.8174 ± 0.0116	0.0222 ± 0.0042	0.8125 ± 0.0159	0.9673 ± 0.0076	0.8090 ± 0.0152
13	CU-R18-MM-A (bs32, lr1e-3, wd1e-4)	1	0.7860 ± 0.0000	0.0262 ± 0.0000	0.8096 ± 0.0000	0.9558 ± 0.0000	0.8047 ± 0.0000
14	CU-R50-CE-R (bs32, lr1e-3, wd1e-4)	3	0.7830 ± 0.0205	1.4882 ± 0.0400	0.8067 ± 0.0158	0.9558 ± 0.0035	0.8050 ± 0.0154
15	CU-R18-CE-R (bs32, lr1e-3, wd1e-4)	1	0.7867 ± 0.0000	1.5035 ± 0.0000	0.8029 ± 0.0000	0.9538 ± 0.0000	0.7994 ± 0.0000
16	CU-R18-MM-AW (bs32, lr1e-3, wd1e-4)	1	0.7658 ± 0.0000	0.0283 ± 0.0000	0.7865 ± 0.0000	0.9606 ± 0.0000	0.7828 ± 0.0000
17	CU-R34-MM-AW (bs32, lr1e-3, wd1e-4)	1	0.7415 ± 0.0000	0.0311 ± 0.0000	0.7808 ± 0.0000	0.9558 ± 0.0000	0.7771 ± 0.0000
18	PT-R50-FR-CE-A (bs32, lr1e-4, wd1e-4)	1	0.8358 ± 0.0000	2.1597 ± 0.0000	0.7702 ± 0.0000	0.9548 ± 0.0000	0.7630 ± 0.0000
19	CU-R50-MM-AW (bs32, lr1e-3, wd1e-4)	3	0.7549 ± 0.0462	0.0309 ± 0.0075	0.7673 ± 0.0281	0.9567 ± 0.0019	0.7654 ± 0.0282
20	PT-R18-FR-CE-A (bs32, lr1e-4, wd1e-4)	1	0.6728 ± 0.0000	1.9048 ± 0.0000	0.7519 ± 0.0000	0.9433 ± 0.0000	0.7453 ± 0.0000

Table 7. ResNet configuration ranking by Kaggle Score (Test Accuracy) (mean ± std across seeds). PT = pre-trained (ImageNet), CU = custom (from scratch), FR = frozen backbone; CE = cross-entropy, MM = multi-margin; A = Adam, AW = AdamW, R = RMSprop.

Rank	Configuration	Seeds	Test Acc
1	PT-R18-CE-A (bs32, lr1e-4, wd1e-4)	5	0.9445 ± 0.0038
2	PT-R50-CE-A (bs32, lr1e-4, wd1e-4)	5	0.9440 ± 0.0071
3	PT-R50-CE-A (bs16, lr5e-5, wd1e-3)	5	0.9426 ± 0.0036
4	PT-R50-CE-R (bs32, lr1e-4, wd1e-4)	3	0.9397 ± 0.0046
5	PT-R50-MM-A (bs32, lr1e-4, wd1e-4)	3	0.9367 ± 0.0042
6	PT-R18-MM-A (bs32, lr1e-4, wd1e-4)	1	0.9238 ± 0.0000
7	PT-R18-CE-R (bs32, lr1e-4, wd1e-4)	1	0.9231 ± 0.0000
8	CU-R18-CE-A (bs32, lr1e-3, wd1e-4)	5	0.8923 ± 0.0075
9	CU-R50-CE-A (bs32, lr1e-3, wd1e-4)	5	0.8902 ± 0.0076
10	CU-R34-CE-A (bs32, lr1e-3, wd1e-4)	5	0.8865 ± 0.0050
11	CU-R50-CE-A (bs16, lr5e-4, wd1e-3)	5	0.8862 ± 0.0037
12	CU-R50-CE-R (bs32, lr1e-3, wd1e-4)	3	0.8241 ± 0.0175
13	CU-R50-MM-A (bs32, lr1e-3, wd1e-4)	3	0.8146 ± 0.0114
14	CU-R18-MM-A (bs32, lr1e-3, wd1e-4)	1	0.8062 ± 0.0000
15	CU-R18-CE-R (bs32, lr1e-3, wd1e-4)	1	0.8015 ± 0.0000
16	PT-R18-FR-CE-A (bs32, lr1e-4, wd1e-4)	1	0.7838 ± 0.0000
17	CU-R34-MM-AW (bs32, lr1e-3, wd1e-4)	1	0.7808 ± 0.0000
18	CU-R50-MM-AW (bs32, lr1e-3, wd1e-4)	3	0.7787 ± 0.0303
19	CU-R18-MM-AW (bs32, lr1e-3, wd1e-4)	1	0.7754 ± 0.0000
20	PT-R50-FR-CE-A (bs32, lr1e-4, wd1e-4)	1	0.7731 ± 0.0000

The results reveal a clear hierarchy between pre-trained and custom-trained models. All top-three configurations by validation accuracy employ ImageNet pre-trained weights with cross-entropy loss and Adam, achieving between 94.6% and 95.0% validation accuracy. PT-R50-CE-A leads at 0.9498 ± 0.0100 with F1 macro 0.9499 ± 0.0105, marginally ahead of its lower learning rate variant (0.9483 ± 0.0063) and PT-R18-CE-A (0.9462 ± 0.0086).

Training accuracy approaches near-perfect values across top configurations (e.g., 0.9957 ± 0.0030 for rank-1), which might initially suggest overfitting. However, deep networks are known to operate in a regime where near-complete training set memorisation does not necessarily indicate poor generalisation [15], and the consistently high validation accuracy and F1 scores corroborate this behaviour.

The gap to custom-trained models is substantial: the best from-scratch configuration, CU-R50-CE-A (0.8925 ± 0.0141), falls 5.7 percentage points below rank 1. Deeper custom architectures offer no consistent advantage over ResNet-18, indicating that additional depth provides limited benefit without pre-trained representations within the training budget. Feature-frozen pre-trained models (PT-R50-FR-CE-A: 77.0%, PT-R18-FR-CE-A: 75.2%) perform worst among all pre-trained variants, confirming that full fine-tuning of the backbone is essential for this 75-class fine-grained task.

Multi-margin loss consistently underperforms cross-entropy across all architectures and training regimes; the near-zero validation losses it produces reflect its bounded nature and are not comparable to cross-entropy values. R configurations show higher variance and generally trail their A counterparts. Kaggle test scores mirror the validation ranking, with PT-R18-CE-A leading on test accuracy (0.9445 ± 0.0038) despite ranking third by validation, suggesting marginally better generalisation from the shallower architecture.

4.4 Comparative Analysis

Performance across all families is governed by two factors: spatial inductive bias and pretrained representations. MLPs establish the lower bound (F1 = 0.344); the gap to the best from-scratch CNN (AlexNetOpt, F1 = 0.863) quantifies the value of convolutional feature extraction alone.

Transfer learning is the single most impactful factor. Pretrained CNNs surpass their from-scratch counterparts by +0.048 F1 despite training only the classifier head; pretrained ResNets with full fine-tuning gain +0.058 F1 over their custom equivalents. Feature-frozen ResNet variants (0.752–0.770) confirm that full backbone fine-tuning is essential for this 75-class fine-grained task.

CE+A is dominant across every family and regime. MM consistently underperforms CE, with a single inversion in VGG16Opt scratch, likely an artefact of the underfitting regime. R trails A in all pretrained configurations, with higher variance in the ResNet family. BatchNorm is critical for from-scratch training, yielding a +0.136 F1 gain on AlexNet, the largest single architectural modification in this study. Model depth provides no consistent benefit without pretrained representations: custom ResNet-50 and -34 do not reliably outperform ResNet-18, mirroring VGG16 scratch underperforming AlexNetOpt. Pretrained models break this pattern, as ImageNet representations allow additional depth to be beneficial.

On the held-out test set, pretrained ResNets achieve the highest accuracy (PT-R18-CE-A, 0.9445 ± 0.0038), marginally ahead of the best pretrained CNN (VGG16Pre, 0.9143 ± 0.0050). In both families, validation stability does not guarantee superior generalisation: PT-R18-CE-A leads on test despite ranking third on validation, and VGG16Pre outperforms AlexNetPre on test despite the latter showing lower variance across seeds.

5 Conclusion

This work compared MLP, CNN, and ResNet architectures for 75-class butterfly species classification. MLPs prove inadequate (F1 = 0.344), confirming that spatial structure is indispensable for fine-grained visual tasks. From-scratch CNNs and ResNets achieve substantially higher performance, with BatchNorm and residual connections as key enablers. Transfer learning is the single most impactful factor: pretrained CNNs (VGG16Pre CE+A, F1 = 0.911) and ResNets (PT-R50-CE-A, F1 = 0.9499) surpass from-scratch counterparts by large margins, with feature-frozen variants confirming that full backbone fine-tuning is essential. CE+A is the dominant combination across all families and regimes. On the held-out test set, PT-R18-CE-A (0.9445 ± 0.0038) emerges as the best model, recommending a pretrained ResNet-18 for the Science Museum's deployment. The study is limited by the small dataset size and the absence of systematic augmentation ablation; future work could explore modern architectures (e.g. EfficientNet) and ensemble strategies.

References

1. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition (2015), https://arxiv.org/abs/1512.03385
2. He, K., Zhang, X., Ren, S., Sun, J.: Delving deep into rectifiers: Surpassing human-level performance on imagenet classification (2015), https://arxiv.org/abs/1502.01852
3. Ioffe, S., Szegedy, C.: Batch normalization: Accelerating deep network training by reducing internal covariate shift (2015), https://arxiv.org/abs/1502.03167
4. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization (2015), https://arxiv.org/abs/1412.6980
5. Krizhevsky, A., Sutskever, I., Hinton, G.E.: Imagenet classification with deep convolutional neural networks. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 25 (2012)
6. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. Proceedings of the IEEE **86**(11), 2278–2324 (1998)
7. Loshchilov, I., Hutter, F.: Sgdr: Stochastic gradient descent with warm restarts (2017), https://arxiv.org/abs/1608.03983
8. Maintainers of Torchvision: TorchVision: PyTorch's Computer Vision Library (2016), https://github.com/pytorch/vision, gitHub repository. ImageNet normalisation statistics discussed at https://github.com/pytorch/vision/pull/1965
9. Nair, V., Hinton, G.E.: Rectified linear units improve restricted boltzmann machines. In: Proceedings of the 27th International Conference on Machine Learning (ICML). pp. 807–814 (2010)
10. Pan, S.J., Yang, Q.: A survey on transfer learning. IEEE Transactions on Knowledge and Data Engineering **22**(10), 1345–1359 (2010)
11. Shorten, C., Khoshgoftaar, T.M.: A survey on image data augmentation for deep learning. Journal of Big Data **6**(1), 1–48 (2019)
12. Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition (2014), https://arxiv.org/abs/1409.1556
13. Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., Wojna, Z.: Rethinking the inception architecture for computer vision (2016), https://arxiv.org/abs/1512.00567
14. Yosinski, J., Clune, J., Bengio, Y., Lipson, H.: How transferable are features in deep neural networks? In: Advances in Neural Information Processing Systems (NeurIPS). vol. 27 (2014)
15. Zhang, C., Bengio, S., Hardt, M., Recht, B., Vinyals, O.: Understanding deep learning requires rethinking generalization (2017), https://arxiv.org/abs/1611.03530